PayWise AI – Product Requirements Document (PRD)

Version: 1.0 (one-day hackathon build) | **Status:** Final for build | **Event:** PayPal Hackathon | **Team:** 4 engineers

---

1. Overview

PayWise AI is a payment-assurance layer on top of PayPal. It checks a payment **before** it is sent, tracks it **after** it is sent, and explains in plain language **why** it is stuck and **what to do next**. Tagline: *Prevent. Detect. Resolve.*

2. Problem Statement

Digital payments fail or stall for reasons users do not understand: a mistyped recipient email, a recipient with no PayPal account, an unverified recipient, insufficient funds, a recipient who must manually accept the payment, or a bank-funded (eCheck) payment that takes days to clear. PayPal shows a status label such as *Unclaimed* or *Pending*, but not the cause, the risk, or the fix. Senders lose time and trust, and recipients lose money or wait without knowing why.

3. Goals and Non-Goals

Goals

- G1. Surface payment risks before money moves (pre-flight check).
- G2. Track every payment through its lifecycle using real PayPal Sandbox events.
- G3. Explain each issue in plain language with a specific next action for sender or recipient.
- G4. Demonstrate all six target problems end to end in a stable live demo.

Non-Goals (explicitly out of scope for the one-day build)

- Production money movement, KYC, or compliance workflows.
- Mobile apps, multi-language UI, multi-currency conversion engine.
- Fraud scoring models trained on data; we use transparent rules.
- Elastic search, admin back office, refunds and disputes handling.

4. Target Users and Personas

Persona	Description	Key need
Sender (primary)	Individual or small business paying someone by email	Know if the payment will succeed, and what to do if it does not
Recipient (secondary)	Person receiving funds	Understand why money is unclaimed or pending and how to receive it
Judge / Evaluator	Hackathon reviewer	See a clear problem, working PayPal integration, and a real AI value-add in minutes

5. The Six Problems Solved

#	Problem	Detection signal	Plain-language message (example)	Recommended action
1	Wrong recipient email	Invalid format, disposable domain, close match to a known recipient (typo distance), or first-time recipient	"This email looks similar to *sam@gmail.com*. Did you mean that?"	Confirm or correct the email before sending
2	Recipient has no PayPal account	Email not linked to a PayWise profile; payout later reports *Unclaimed*	"This person may not have a PayPal account yet. Funds will wait up to 30 days."	Ask recipient to sign up with this exact email; sender can cancel unclaimed payment
3	Incomplete verification	Recipient profile flagged unverified, or payout *Held/Blocked*	"The recipient's account needs verification before it can receive funds."	Recipient completes verification in PayPal

#	Problem	Detection signal	Plain-language message (example)	Recommended action
4	Insufficient balance	Amount plus fee estimate greater than sender balance	"You are short by 12.40 USD."	Add funds or lower the amount
5	Manual acceptance required	Pending reason indicates the recipient must accept payment	"The recipient's settings require them to accept this payment manually."	Recipient accepts in PayPal; sender can nudge
6	eCheck / bank delay	Pending reason eCheck on a bank-funded payment	"This payment is funded by a bank transfer and typically clears in 3-5 business days."	Wait, or pay by card/balance next time

This table is the single source of truth for the rule engine, the explanation templates, and the demo scripts.

6. Core User Journeys

1. **Send with pre-flight:** Sender logs in, enters recipient email and amount. Live checks appear as they type. A risk summary (Low / Medium / High) shows each check with pass, warn or fail. Sender confirms and pays.
2. **Track and understand:** The dashboard shows the payment moving from Created to Pending to Completed, Unclaimed or Failed. Status changes arrive in real time. Each non-success status has an *Explain* panel with cause and next steps.
3. **Resolve:** The sender sees recommended actions, can copy a ready-to-send message to the recipient, and receives an in-app alert when the status changes.

7. Functional Requirements

Priority: **P0** = must ship, **P1** = ship if time allows, **P2** = stretch.

ID	Requirement	Priority
FR-1	Email/password sign-up and login (Supabase Auth); sender and recipient roles	P0
FR-2	Payment form: recipient email, amount, currency, note	P0
FR-3	Pre-flight endpoint returns risk level and per-check results for problems 1-6	P0
FR-4	Live recipient validation (debounced) in the form	P0
FR-5	Create a real PayPal Sandbox payment and store its transaction record	P0
FR-6	Receive and verify PayPal webhooks; update transaction status and log raw events	P0
FR-7	Dashboard: balance, recent payments, status overview, real-time updates	P0
FR-8	Explain panel: Gemini-written explanation with template fallback	P0
FR-9	In-app alerts (notification list and unread badge)	P0
FR-10	Transaction history table with filter by status	P1
FR-11	Pending Status View with timeline per transaction	P1
FR-12	Scenario Simulator: trigger each of the six cases on demand for the demo	P0
FR-13	Email notification for status changes	P2
FR-14	Currency mismatch warning	P2
FR-15	Landing page with product pitch and navigation	P1

8. Non-Functional Requirements

- **Performance:** pre-flight response under 800 ms (rules only); explanation under 4 s, with instant template fallback if Gemini is slow or fails.

- **Reliability:** webhook handler is idempotent and returns 200 quickly; PayPal retries cannot create duplicates.
- **Security:** Row Level Security on all tables; JWT validated on every API call; secrets only in environment variables; emails masked before being sent to Gemini; PayPal webhook signature verified.
- **Usability:** a first-time user completes a pre-flight check in under 30 seconds; all explanations at roughly a 7th-grade reading level.
- **Demo safety:** the app clearly labels Sandbox mode; no real money or real credentials are used.

9. AI Behavior Requirements

- Deterministic rules decide the issue and risk level. Gemini only **explains** the structured result; it never decides it.
- Output is structured JSON: `summary`, `cause`, `next_steps[]`, `message_to_recipient`.
- Explanations must not claim certainty beyond the signal (use wording such as "may" when a signal is inferred).
- If Gemini fails or times out, the template for that issue code is shown instead.

10. Key Assumptions and Constraints (verify in the first hour)

1. PayPal Sandbox Payouts is enabled on the sandbox business account. If it is not, the fallback is the Orders v2 flow plus the Scenario Simulator.
2. PayPal has no public API to check whether an arbitrary email owns an account. "Has PayPal" is inferred from PayWise profiles and confirmed after the fact by the *Unclaimed* status. The UI labels this as a prediction.
3. Sender balance is a **simulated wallet** stored in Supabase for the sandbox demo, because PayPal does not expose arbitrary users' balances.
4. eCheck and manual-acceptance cases are reproduced in the demo through the Scenario Simulator using real PayPal event payload shapes.

11. Success Metrics (for the demo and judging)

- All six scenarios demonstrated end to end in under 5 minutes.

- At least one real Sandbox payment completes with a real webhook updating the dashboard.
- Pre-flight catches every seeded bad-input case (6 of 6).
- Zero unhandled errors during the live demo; app deployed and reachable.

12. Risks and Mitigations

Risk	Likelihood	Mitigation
Payouts not enabled in sandbox	Medium	Check in hour 1; fall back to Orders v2 plus Simulator
Webhooks cannot reach local machine	High	Develop against deployed Render URL or a tunnel; polling fallback
Render cold start during demo	Medium	Health-check pinger; warm up before presenting
Gemini latency or quota	Medium	Template fallback; cache by issue code
Scope creep	High	P0-only until integration is green; P1/P2 after

13. Release Scope Summary

Ship (P0): auth, payment form, pre-flight engine, real sandbox payment, webhook tracking, dashboard with real-time updates, Gemini explanations with fallback, alerts, Scenario Simulator.

If time allows (P1/P2): history table, pending timeline, landing page polish, email alerts, currency warning.

14. Demo Script (5 minutes)

1. 30 s: problem and tagline.
2. 60 s: pre-flight catching a mistyped email and an insufficient balance.
3. 90 s: a real sandbox payment to an unregistered email becomes *Unclaimed*; the dashboard updates live; Explain panel shows cause and fix.

4. 90 s: Scenario Simulator shows manual acceptance, eCheck delay and held verification with explanations.
5. 30 s: architecture slide and wrap-up.